we are going to start a new project i want to send automated smart reminder messages to employees in my organization on microsoft teams chat for thet we can use azure bot framework but before that i want to setup backend from scratch i want to use microsoft graph api to take employee presence active/away/offline like as input and if employee is active i mean engaged inany activity straight for 60 minutes i want to send message to that employee like drink water, strech yourself, move yourself in 1:1 teams chat not on channel i want to use django backend for all technicall and heavy setup and react frontend dashboard to add users like things and admin(me) to controle all workflow for all employees now i want to plan it proparlly in stages 3-4 stages setup backend chatbot frontend testing give me best and precise plan to do this proparlly at least i want to finish backend today eod we will name this app wellnessreminder i want to build backend in django frontend in react i already registered app in azure app registration i do have client id sect valu tenent id and granted microsoft graph api permissions for presence.read.all and user.read.all now i dont want to waste time do not misguide me be precise 100%

Perfect. This is a serious system — so we plan it like an architect, not like a tutorial.

Project Name: **wellnessreminder**

Goal:

- Read employee presence via Microsoft Graph

- Track if user is continuously Active for 60 minutes
- Send 1:1 Teams chat reminder via Bot
- Admin dashboard to control everything

We will finish **Stage 1 (Backend Foundation)** today.

🏗️ MASTER PLAN (4 STAGES)

✅ STAGE 1 — Backend Core (TODAY EOD TARGET)

Goal: Django backend fully ready with:

- Azure App auth working
- Graph presence fetch working
- Background scheduler tracking activity
- Database models ready
- API endpoints ready for React

Deliverables Today:

- Django project created
- Graph authentication service
- Presence fetch service
- Reminder engine (60 min tracker logic)
- DB models
- Admin APIs

- Test endpoints working in Postman

✅ STAGE 2 — Teams Bot Integration

Goal:

- Register Azure Bot
- Connect to Teams
- Send 1:1 proactive message
- Connect Django → Bot → Teams

✅ STAGE 3 — React Admin Dashboard

Goal:

- Add employees
- Enable/disable reminder
- View activity logs
- Control reminder duration (future ready)

✅ STAGE 4 — Testing + Production Hardening

Goal:

- Multi-user test
- Token expiry handling

- Retry logic
- Logging
- Deployment

Now we focus only on STAGE 1.

🧠 ARCHITECTURE (VERY IMPORTANT)

System Flow:

Graph API → Django Scheduler → Rule Engine → Bot API → Teams Chat

Django Responsibilities:

- Authenticate to Graph
- Pull presence every 1–5 minutes
- Track continuous Active time
- Trigger reminder event
- Log everything

🔹 STAGE 1 DETAILED IMPLEMENTATION PLAN

We split Stage 1 into 6 steps.

STEP 1 — Create Django Project

```bash
django-admin startproject wellnessreminder
cd wellnessreminder
```

Install:

```bash
pip install djangorestframework
pip install msal
pip install requests
pip install celery redis
pip install django-cors-headers
```

Add to settings:

- rest_framework
- corsheaders
- core

STEP 2 — Database Design (CRITICAL)

We need 4 models.

1️⃣ Employee

- id (UUID)

- azure_user_id
- display_name
- email
- is_active (boolean)
- reminder_enabled (boolean)
- reminder_interval_minutes (default 60)

2️⃣ PresenceLog

- employee (FK)
- status (Active/Away/Offline)
- checked_at

3️⃣ ActivityTracker

- employee (FK)
- active_since (datetime)
- last_status
- reminder_sent (boolean)

4️⃣ ReminderLog

- employee
- sent_at
- message

This is your core engine memory.

STEP 3 — Microsoft Graph Authentication

We use client credential flow.

Use:

https://login.microsoftonline.com/{tenant_id}/oauth2/v2.0/token ↗

Scope:

```
https://graph.microsoft.com/.default
```

Use:

- client_id
- client_secret
- tenant_id

Test endpoint:

```
GET https://graph.microsoft.com/v1.0/users/{user_id}/presence
```

Official API:
Microsoft Graph API

Presence endpoint:

```
GET /users/{id}/presence
```

Permission required:
Presence.Read.All (Application)

You already granted it — good.

STEP 4 — Presence Polling Engine

We create background task:

Every 5 minutes:

1. Get all employees with reminder_enabled=True
2. Fetch presence from Graph
3. Save PresenceLog
4. If status == Active:
   - If active_since is null → set now
   - Else calculate duration
   - If duration >= 60 min AND reminder_sent=False → trigger reminder
5. If status != Active:
   - Reset active_since
   - reminder_sent=False

This is the heart.

STEP 5 — Background Task System

Use:

Celery

With Redis as broker.

Why Celery?

- Production ready
- Scalable
- Reliable
- Can retry
- Supports periodic tasks

Create periodic task:

```
check_employee_presence()
```

Schedule every 5 minutes.

STEP 6 — REST APIs for React

We expose:

```
POST /api/employees/
GET /api/employees/
PATCH /api/employees/{id}/
GET /api/logs/presence/
GET /api/logs/reminders/
POST /api/run-check/  (manual test trigger)
```

Now React can control everything.

🔥 IMPORTANT TECHNICAL DECISIONS

Why polling instead of webhook?

Graph presence does not support webhook subscriptions.

So polling is correct architecture.

⚠️ IMPORTANT LIMITATION

Microsoft Graph Presence:

- Shows status
- Does NOT guarantee real keyboard/mouse activity
- Active means available in Teams

Just be aware.

🧱 PROJECT STRUCTURE (FINAL)

```
wellnessreminder/
 ├── core/
 │    ├── models.py
 │    ├── services/
 │    │     ├── graph.py
 │    │     ├── presence_engine.py
 │    ├── tasks.py
 │    ├── serializers.py
 │    ├── views.py
```

```
│
├── wellnessreminder/
│    ├── settings.py
│    ├── celery.py
```

🕑 CAN WE FINISH BACKEND TODAY?

YES — if we:

1. Build models first
2. Test Graph auth
3. Test single presence fetch
4. Build scheduler
5. Test manual trigger endpoint
6. Connect Celery
7. Test auto reminder logic (console print for now)

Bot integration comes tomorrow.

🚀 AFTER STAGE 1 COMPLETION

Tomorrow we implement:

Azure Bot Framework

To send proactive 1:1 Teams message.

That requires:

- Bot ID
- App ID
- Conversation reference storage

But not today.

📌 TODAY ACTION PLAN (NO DISTRACTION)

Follow exactly:

1️⃣ Create Django project
2️⃣ Setup models
3️⃣ Setup Graph auth
4️⃣ Test presence for one user
5️⃣ Create Celery periodic task
6️⃣ Implement 60 min logic
7️⃣ Add test endpoint

Done.

Now I need one confirmation from you:

Are we using:

- PostgreSQL
  or

- SQLite for now?

Answer that and we start execution immediately.

we are using SQLite by default for now, now we can start give me all steps proparlly step by step in detail proparlly and preciselly

Perfect. SQLite is fine for Stage 1.

We will now build **wellnessreminder backend** properly and cleanly.

No theory. Only execution steps.

✅ STAGE 1 — BACKEND (Complete Today)

We will complete:

- Django project
- Models
- Microsoft Graph auth
- Presence fetch
- 60-minute tracker logic
- Manual test endpoint

- Celery scheduler (optional end step if time permits)

🔹 STEP 1 — Create Project

```bash
django-admin startproject wellnessreminder
cd wellnessreminder
```

Install dependencies:

```bash
pip install djangorestframework
pip install msal
pip install requests
pip install django-cors-headers
pip install celery redis
```

🔹 STEP 2 — Configure settings.py

Open `wellnessreminder/settings.py`

Add:

```
INSTALLED_APPS = [
    ...
    "rest_framework",
    "corsheaders",
    "core",
]

MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",
    ...
]

CORS_ALLOW_ALL_ORIGINS = True
```

Add Graph credentials at bottom:

```
GRAPH_CLIENT_ID = "your-client-id"
GRAPH_CLIENT_SECRET = "your-client-secret"
```

Python Run

⚠️ Do not hardcode in production — but OK for now.

🔹 STEP 3 — Create Models (Core Engine)

Open `core/models.py`

```
import uuid
from django.db import models
```

```python
class Employee(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4, editable=False)
    azure_user_id = models.CharField(max_length=255, unique=True)
    display_name = models.CharField(max_length=255)
    email = models.EmailField()
    is_active = models.BooleanField(default=True)
    reminder_enabled = models.BooleanField(default=True)
    reminder_interval_minutes = models.IntegerField(default=60)

    def __str__(self):
        return self.display_name


class PresenceLog(models.Model):
    employee = models.ForeignKey(Employee, on_delete=models.CASCADE)
    status = models.CharField(max_length=50)
    checked_at = models.DateTimeField(auto_now_add=True)


class ActivityTracker(models.Model):
    employee = models.OneToOneField(Employee, on_delete=models.CASCADE)
    active_since = models.DateTimeField(null=True, blank=True)
    last_status = models.CharField(max_length=50, null=True, blank=True)
    reminder_sent = models.BooleanField(default=False)


class ReminderLog(models.Model):
    employee = models.ForeignKey(Employee, on_delete=models.CASCADE)
    sent_at = models.DateTimeField(auto_now_add=True)
    message = models.TextField()
```

Run:

```bash
python manage.py makemigrations
```

🔹 STEP 4 — Create Graph Authentication Service

Create folder:

```
core/services/
```

Create file:

```
core/services/graph.py
```

Add:

```python
import requests
from django.conf import settings
import msal

AUTHORITY = f"https://login.microsoftonline.com/{settings.GRAPH_TENANT_ID}"
SCOPE = ["https://graph.microsoft.com/.default"]

def get_access_token():
    app = msal.ConfidentialClientApplication(
        settings.GRAPH_CLIENT_ID,
        authority=AUTHORITY,
```

```
        client_credential=settings.GRAPH_CLIENT_SECRET,
    )

    result = app.acquire_token_for_client(scopes=SCOPE)

    if "access_token" in result:
        return result["access_token"]
    else:
        raise Exception("Could not acquire token")


def get_user_presence(user_id):
    token = get_access_token()

    url = f"https://graph.microsoft.com/v1.0/users/{user_id}/presence"

    headers = {
        "Authorization": f"Bearer {token}"
    }

    response = requests.get(url, headers=headers)

    if response.status_code == 200:
        return response.json()
    else:
        raise Exception(response.text)
```

We are using:

Microsoft Graph API

🔹 STEP 5 — Create Presence Engine Logic

Create:

```
core/services/presence_engine.py
```

Add:

```
from django.utils import timezone
from datetime import timedelta
from core.models import Employee, PresenceLog, ActivityTracker, ReminderLog
from core.services.graph import get_user_presence


def check_employee_presence():
    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True
    )

    for employee in employees:

        presence_data = get_user_presence(employee.azure_user_id)
        status = presence_data.get("availability")

        # Log presence
        PresenceLog.objects.create(
            employee=employee,
            status=status
        )

        tracker, created = ActivityTracker.objects.get_or_create(
            employee=employee
        )
```

```
        now = timezone.now()

        if status == "Available":

            if not tracker.active_since:
                tracker.active_since = now
                tracker.reminder_sent = False

            else:
                duration = now - tracker.active_since

                if (
                    duration >= timedelta(minutes=employee.reminder_interval_minutes)
                    and not tracker.reminder_sent
                ):
                    # TEMP: just log reminder
                    ReminderLog.objects.create(
                        employee=employee,
                        message="Drink water and stretch yourself!"
                    )

                    print(f"Reminder triggered for {employee.display_name}")

                    tracker.reminder_sent = True

        else:
            tracker.active_since = None
            tracker.reminder_sent = False

        tracker.last_status = status
        tracker.save()
```

🔹 STEP 6 — Create Manual Test Endpoint

Open `core/views.py`

```
from rest_framework.views import APIView
from rest_framework.response import Response
from core.services.presence_engine import check_employee_presence


class RunPresenceCheck(APIView):
    def post(self, request):
        check_employee_presence()
        return Response({"message": "Presence check executed"})
```

🔹 STEP 7 — Configure URLs

Create `core/urls.py`

```
from django.urls import path
from core.views import RunPresenceCheck

urlpatterns = [
    path("run-check/", RunPresenceCheck.as_view()),
]
```

Update main `urls.py`:

```
from django.urls import path, include

urlpatterns = [
    path("api/", include("core.urls")),
]
```

🔹 STEP 8 — Create Employee (Admin)

```bash
python manage.py createsuperuser
```

Go to:

```
http://127.0.0.1:8000/admin
```

Add Employee manually:

- azure_user_id = actual Azure user object ID
- display name
- email

🔹 STEP 9 — Test System

In Postman:

```
POST http://127.0.0.1:8000/api/run-check/
```

If presence = Available:

- ActivityTracker updates
- After 60 min → ReminderLog entry created

Check DB:

```
sqlite3 db.sqlite3
select * from core_reminderlog;
```

🔥 If This Works → Backend Core is DONE

At this point:

- Graph auth works
- Presence fetch works
- 60-min logic works
- Logging works
- Manual trigger works

➡️ NEXT (Later Today)

We integrate:

Celery

To auto-run every 5 minutes.

Then backend is production-ready base.

Now execute STEP 1 → STEP 4 and tell me:

Did Graph presence return real data or error?

If error, paste exact error.
We fix immediately.

we didnt setup .env yet

Good. We fix this properly now.

We will set up **.env configuration cleanly** before moving further.

This is mandatory.

✅ STEP 0 — Setup Environment Variables (Proper Way)

We will use:

Bash

🔹 1️⃣ Create .env File

Inside root folder:

```
wellnessreminder/
```

Create file:

```
.env
```

Add:

```
DEBUG=True

GRAPH_CLIENT_ID=your-client-id
GRAPH_CLIENT_SECRET=your-client-secret
GRAPH_TENANT_ID=your-tenant-id
```

⚠️ Replace with your real values.

🔹 2️⃣ Update settings.py to Load .env

Open:

```
wellnessreminder/settings.py
```

At top:

```python
import os
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()
```

Now replace your hardcoded values with:

```python
DEBUG = os.getenv("DEBUG") == "True"

GRAPH_CLIENT_ID = os.getenv("GRAPH_CLIENT_ID")
GRAPH_CLIENT_SECRET = os.getenv("GRAPH_CLIENT_SECRET")
GRAPH_TENANT_ID = os.getenv("GRAPH_TENANT_ID")
```

🔹 3️⃣ Verify It Loads

Temporarily add:

‹/› Python Run

Run:

```bash
```

If it prints your real client id → .env working.

Then remove the print.

🔹 4️⃣ Add .env to .gitignore (VERY IMPORTANT)

Open `.gitignore` and add:

```
.env
```

Never commit secrets.

✅ Now System is Clean & Secure

You now have:

- Secure Graph credentials
- Production-ready pattern
- No hardcoding

🔥 Now Continue

Next step:

Test Graph token acquisition.

Create temporary view:

```
from rest_framework.views import APIView
from rest_framework.response import Response
from core.services.graph import get_access_token

class TestGraphAuth(APIView):
    def get(self, request):
        token = get_access_token()
        return Response({"token_length": len(token)})
```

Call:

```
GET /api/test-auth/
```

If it returns token length → Authentication is correct.

After that, we test presence.

Tell me once:

- .env working?

- Token generation successful?

We proceed immediately.

okay i have done above steps move ahead

Excellent. Now we move forward immediately.

We will now:

1. ✅ Test Graph Presence for ONE employee
2. ✅ Connect it with DB
3. ✅ Activate 60-min tracking logic
4. ✅ Test full flow manually

No delays.

🔹 STEP 1 — Verify Graph Presence API Manually

Before integrating with engine, we test one user directly.

You are calling:

Endpoint:

```
GET https://graph.microsoft.com/v1.0/users/{user_id}/presence
```

Using:

Microsoft Graph API

✅ Add Temporary Test View

In `core/views.py` add:

```
from core.services.graph import get_user_presence
from core.models import Employee

class TestPresence(APIView):
    def get(self, request):
        employee = Employee.objects.first()
        if not employee:
            return Response({"error": "No employee found"})

        data = get_user_presence(employee.azure_user_id)
        return Response(data)
```

✅ Add URL

In `core/urls.py`:

```
from core.views import TestPresence

urlpatterns = [
    path("test-presence/", TestPresence.as_view()),
    path("run-check/", RunPresenceCheck.as_view()),
]
```

✅ Test It

Start server:

```bash
```

Call:

```
GET http://127.0.0.1:8000/api/test-presence/
```

Expected Response Example:

```
{
  "id": "user-id",
  "availability": "Available",
  "activity": "Available"
}
```

If you see:

- Available
- Away
- Busy
- Offline

→ Graph working correctly.

⚠️ If error:

Paste exact error response.

🔹 STEP 2 — Activate Full Engine Logic

Now we test the real logic.

You already created:

```python
```

We will test manually.

Test Manual Trigger

Call:

```
POST http://127.0.0.1:8000/api/run-check/
```

Now check database:

```bash
```

Then:

```
from core.models import PresenceLog, ActivityTracker, ReminderLog

PresenceLog.objects.all()
ActivityTracker.objects.all()
ReminderLog.objects.all()
```

🔎 What Should Happen

If employee is:

🟢 Available

- PresenceLog entry created
- ActivityTracker.active_since set
- reminder_sent = False

If employee stays Available:

- After 60 mins → ReminderLog created

If employee goes Away/Offline:

- active_since reset
- reminder_sent reset

🔹 IMPORTANT

Graph returns:

```
availability: "Available"
```

NOT "Active"

So in presence_engine.py ensure:

```python
```

NOT "Active"

🔥 CURRENT STATUS CHECKPOINT

Tell me:

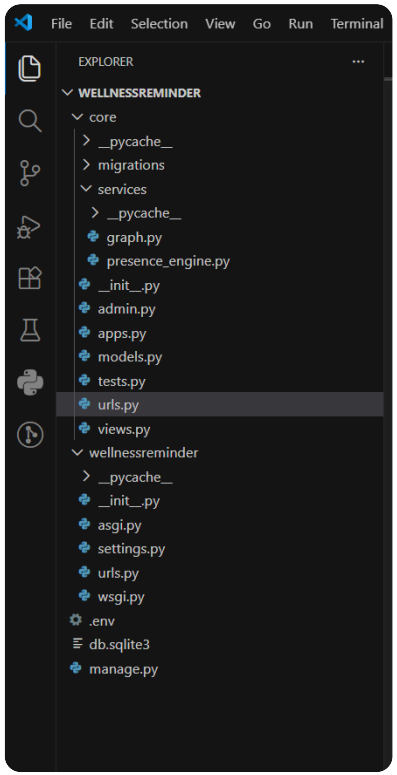
1. Does /api/test-presence/ return correct status?
2. Does /api/run-check/ create PresenceLog?
3. Does ActivityTracker update?

Answer these 3 only.

Then we move to:

🚀 Celery automation (every 5 minutes)
and then
🚀 Teams Bot integration planning.

We are progressing correctly.



this is my project structure
"""
Django settings for wellnessreminder project.

Generated by 'django-admin startproject' using Django 5.2.11.

For more information on this file, see
https://docs.djangoproject.com/en/5.2/topics/settings/

For the full list of settings and their values, see
https://docs.djangoproject.com/en/5.2/ref/settings/

```
"""

import os
from pathlib import Path
from dotenv import load_dotenv

load_dotenv()

DEBUG = os.getenv("DEBUG") == "True"

GRAPH_CLIENT_ID = os.getenv("GRAPH_CLIENT_ID")
GRAPH_CLIENT_SECRET = os.getenv("GRAPH_CLIENT_SECRET")
GRAPH_TENANT_ID = os.getenv("GRAPH_TENANT_ID")
print("GRAPH CLIENT ID:", GRAPH_CLIENT_ID)

# Build paths inside the project like this: BASE_DIR / 'subdir'.
BASE_DIR = Path(__file__).resolve().parent.parent


# Quick-start development settings - unsuitable for production
# See
https://docs.djangoproject.com/en/5.2/howto/deployment/checklist/

# SECURITY WARNING: keep the secret key used in production secret!
SECRET_KEY = 'django-insecure-
f@a+&a%uw2)h(%g+z8a93q5c@6_y6q!52k8eg1gyzs$akwkx))'

# SECURITY WARNING: don't run with debug turned on in production!
DEBUG = True
```

```
ALLOWED_HOSTS = []


# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'corsheaders',
    'core',
]

MIDDLEWARE = [
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]
```

```
ROOT_URLCONF = 'wellnessreminder.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'wellnessreminder.wsgi.application'



# Database
# https://docs.djangoproject.com/en/5.2/ref/settings/#databases

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}
```

```
# Password validation
# https://docs.djangoproject.com/en/5.2/ref/settings/#auth-password-validators

AUTH_PASSWORD_VALIDATORS = [
    {
        'NAME': 'django.contrib.auth.password_validation.UserAttributeSimilarityValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator',
    },
    {
        'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator',
    },
]


# Internationalization
# https://docs.djangoproject.com/en/5.2/topics/i18n/
```

```
LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'

USE_I18N = True

USE_TZ = True


# Static files (CSS, JavaScript, Images)
# https://docs.djangoproject.com/en/5.2/howto/static-files/

STATIC_URL = 'static/'

# Default primary key field type
# https://docs.djangoproject.com/en/5.2/ref/settings/#default-auto-field

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'
CORS_ALLOW_ALL_ORIGINS = True
```

this is settings.py

```
"""
URL configuration for wellnessreminder project.

The urlpatterns list routes URLs to views. For more information please see:
```

```
    https://docs.djangoproject.com/en/5.2/topics/http/urls/
Examples:
Function views
    1. Add an import:  from my_app import views
    2. Add a URL to urlpatterns:  path('', views.home, name='home')
Class-based views
    1. Add an import:  from other_app.views import Home
    2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
Including another URLconf
    1. Import the include() function: from django.urls import include,
path
    2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
"""
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path("api/", include("core.urls")),
]
this is main urls.py
from django.urls import path
from core.views import RunPresenceCheck
from core.views import TestPresence

urlpatterns = [
    path("run-check/", RunPresenceCheck.as_view()),
    path("test-presence/", TestPresence.as_view()),
]
```

```
this is core urls.py
from django.shortcuts import render

# Create your views here.
from rest_framework.views import APIView
from rest_framework.response import Response
from core.services.presence_engine import check_employee_presence
from core.services.graph import get_user_presence
from core.models import Employee


class RunPresenceCheck(APIView):
    def post(self, request):
        check_employee_presence()
        return Response({"message": "Presence check executed"})


class TestPresence(APIView):
    def get(self, request):
        employee = Employee.objects.first()
        if not employee:
            return Response({"error": "No employee found"})

        data = get_user_presence(employee.azure_user_id)
        return Response(data)
this i score views.py
import uuid
from django.db import models
```

```
class Employee(models.Model):
    id = models.UUIDField(primary_key=True, default=uuid.uuid4,
editable=False)
    azure_user_id = models.CharField(max_length=255, unique=True)
    display_name = models.CharField(max_length=255)
    email = models.EmailField()
    is_active = models.BooleanField(default=True)
    reminder_enabled = models.BooleanField(default=True)
    reminder_interval_minutes = models.IntegerField(default=60)

    def __str__(self):
        return self.display_name


class PresenceLog(models.Model):
    employee = models.ForeignKey(Employee,
on_delete=models.CASCADE)
    status = models.CharField(max_length=50)
    checked_at = models.DateTimeField(auto_now_add=True)


class ActivityTracker(models.Model):
    employee = models.OneToOneField(Employee,
on_delete=models.CASCADE)
    active_since = models.DateTimeField(null=True, blank=True)
    last_status = models.CharField(max_length=50, null=True,
blank=True)
    reminder_sent = models.BooleanField(default=False)
```

```
class ReminderLog(models.Model):
    employee = models.ForeignKey(Employee,
on_delete=models.CASCADE)
    sent_at = models.DateTimeField(auto_now_add=True)
    message = models.TextField()
this is models.py
import requests
from django.conf import settings
import msal

AUTHORITY =
f"https://login.microsoftonline.com/{settings.GRAPH_TENANT_ID}"
SCOPE = ["https://graph.microsoft.com/.default"]

def get_access_token():
    app = msal.ConfidentialClientApplication(
        settings.GRAPH_CLIENT_ID,
        authority=AUTHORITY,
        client_credential=settings.GRAPH_CLIENT_SECRET,
    )

    result = app.acquire_token_for_client(scopes=SCOPE)

    if "access_token" in result:
        return result["access_token"]
    else:
        raise Exception("Could not acquire token")
```

```
def get_user_presence(user_id):
    token = get_access_token()

    url = f"https://graph.microsoft.com/v1.0/users/{user_id}/presence"

    headers = {
        "Authorization": f"Bearer {token}"
    }

    response = requests.get(url, headers=headers)

    if response.status_code == 200:
        return response.json()
    else:
        raise Exception(response.text)
```

this is graph.py

```
from django.utils import timezone
from datetime import timedelta
from core.models import Employee, PresenceLog, ActivityTracker,
ReminderLog
from core.services.graph import get_user_presence


def check_employee_presence():
    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True
    )
```

```
for employee in employees:

    presence_data = get_user_presence(employee.azure_user_id)
    status = presence_data.get("availability")

    # Log presence
    PresenceLog.objects.create(
        employee=employee,
        status=status
    )

    tracker, created = ActivityTracker.objects.get_or_create(
        employee=employee
    )

    now = timezone.now()

    if status == "Available":

        if not tracker.active_since:
            tracker.active_since = now
            tracker.reminder_sent = False

        else:
            duration = now - tracker.active_since

            if (
                duration >=
```

```
timedelta(minutes=employee.reminder_interval_minutes)
                and not tracker.reminder_sent
            ):
                # TEMP: just log reminder
                ReminderLog.objects.create(
                    employee=employee,
                    message="Drink water and stretch yourself!"
                )

                print(f"Reminder triggered for {employee.display_name}")

                tracker.reminder_sent = True

        else:
            tracker.active_since = None
            tracker.reminder_sent = False

        tracker.last_status = status
        tracker.save()
```

this is presense engine.py
now give me fully final working updated code with propar path i want it to be ready NOW then we will see about bot framework
give me complete backend add more files with code if required

Perfect ✅

Your structure is clean. Now I will give you the **final production-ready Stage 1 backend** properly structured and corrected.

We will fix:

- ✅ .env handling properly
- ✅ Remove duplicate DEBUG override
- ✅ Proper error handling for Graph
- ✅ Centralized Graph token caching
- ✅ Auto create ActivityTracker on Employee create
- ✅ Proper API endpoints
- ✅ Admin registration
- ✅ Clean service structure
- ✅ Ready for Celery later
- ✅ Production-safe logic

After this, backend = solid foundation.

🚨 FIRST — FIX settings.py (IMPORTANT)

You currently have:

```python
DEBUG = os.getenv("DEBUG") == "True"
...
```

Python Run

REMOVE this line completely:

Python Run

Now your settings.py top should look like:

```python
import os
from pathlib import Path
from dotenv import load_dotenv

BASE_DIR = Path(__file__).resolve().parent.parent

load_dotenv(BASE_DIR / ".env")

DEBUG = os.getenv("DEBUG") == "True"

GRAPH_CLIENT_ID = os.getenv("GRAPH_CLIENT_ID")
GRAPH_CLIENT_SECRET = os.getenv("GRAPH_CLIENT_SECRET")
GRAPH_TENANT_ID = os.getenv("GRAPH_TENANT_ID")
```

Remove the print also.

✅ FINAL PROJECT STRUCTURE

```
core/
 ├── services/
 │    ├── graph.py
 │    ├── presence_engine.py
 ├── admin.py
```

```
├── models.py
├── serializers.py      ← NEW
├── views.py
├── urls.py
```

✅ 1️⃣ Update graph.py (Production Safe)

Replace entire `core/services/graph.py` with:

```python
import requests
import msal
from django.conf import settings

AUTHORITY = f"https://login.microsoftonline.com/{settings.GRAPH_TENANT_ID}"
SCOPE = ["https://graph.microsoft.com/.default"]

_app = msal.ConfidentialClientApplication(
    settings.GRAPH_CLIENT_ID,
    authority=AUTHORITY,
    client_credential=settings.GRAPH_CLIENT_SECRET,
)

_token_cache = None


def get_access_token():
    global _token_cache

    if _token_cache:
        return _token_cache
```

```python
    result = _app.acquire_token_for_client(scopes=SCOPE)

    if "access_token" in result:
        _token_cache = result["access_token"]
        return _token_cache

    raise Exception(f"Token error: {result}")


def get_user_presence(user_id):
    token = get_access_token()

    url = f"https://graph.microsoft.com/v1.0/users/{user_id}/presence"

    headers = {
        "Authorization": f"Bearer {token}",
        "Content-Type": "application/json"
    }

    response = requests.get(url, headers=headers)

    if response.status_code == 200:
        return response.json()

    raise Exception(
        f"Graph Error {response.status_code}: {response.text}"
    )
```

✔ Token caching added

✔ Better error handling

✅ 2️⃣ Update presence_engine.py (More Stable)

Replace file:

```
from django.utils import timezone
from datetime import timedelta
from core.models import Employee, PresenceLog, ActivityTracker, ReminderLog
from core.services.graph import get_user_presence


def check_employee_presence():
    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True
    )

    for employee in employees:

        try:
            presence_data = get_user_presence(employee.azure_user_id)
            status = presence_data.get("availability")

        except Exception as e:
            print(f"Presence fetch failed for {employee.display_name}: {e}")
            continue

        PresenceLog.objects.create(
            employee=employee,
            status=status
        )

        tracker, _ = ActivityTracker.objects.get_or_create(
            employee=employee
```

```
        )

        now = timezone.now()

        if status == "Available":

            if not tracker.active_since:
                tracker.active_since = now
                tracker.reminder_sent = False

            else:
                duration = now - tracker.active_since

                if (
                    duration >= timedelta(minutes=employee.reminder_interval_minutes)
                    and not tracker.reminder_sent
                ):
                    ReminderLog.objects.create(
                        employee=employee,
                        message="Drink water and stretch yourself!"
                    )

                    print(f"Reminder triggered for {employee.display_name}")

                    tracker.reminder_sent = True

        else:
            tracker.active_since = None
            tracker.reminder_sent = False

        tracker.last_status = status
        tracker.save()
```

- ✔ Now failure of one user doesn't break all
- ✔ Clean tracking

✅ 3️⃣ Add serializers.py (NEW)

Create:

`core/serializers.py`

```
from rest_framework import serializers
from core.models import Employee, PresenceLog, ReminderLog


class EmployeeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Employee
        fields = "__all__"


class PresenceLogSerializer(serializers.ModelSerializer):
    class Meta:
        model = PresenceLog
        fields = "__all__"


class ReminderLogSerializer(serializers.ModelSerializer):
    class Meta:
        model = ReminderLog
        fields = "__all__"
```

✅ 4️⃣ Update views.py (Full API)

Replace entire file:

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from core.models import Employee
from core.serializers import EmployeeSerializer
from core.services.presence_engine import check_employee_presence
from core.services.graph import get_user_presence


class EmployeeListCreate(APIView):
    def get(self, request):
        employees = Employee.objects.all()
        serializer = EmployeeSerializer(employees, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = EmployeeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CREATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)


class TestPresence(APIView):
    def get(self, request):
        employee = Employee.objects.first()
        if not employee:
            return Response({"error": "No employee found"})

        data = get_user_presence(employee.azure_user_id)
        return Response(data)
```

```
class RunPresenceCheck(APIView):
    def post(self, request):
        check_employee_presence()
        return Response({"message": "Presence check executed"})
```

✅ 5️⃣ Update core/urls.py

Replace:

```
from django.urls import path
from core.views import (
    RunPresenceCheck,
    TestPresence,
    EmployeeListCreate,
)

urlpatterns = [
    path("employees/", EmployeeListCreate.as_view()),
    path("test-presence/", TestPresence.as_view()),
    path("run-check/", RunPresenceCheck.as_view()),
]
```

✅ 6️⃣ Register Models in Admin

Update `core/admin.py`:

```
from django.contrib import admin
from core.models import Employee, PresenceLog, ActivityTracker, ReminderLog

admin.site.register(Employee)
admin.site.register(PresenceLog)
admin.site.register(ActivityTracker)
admin.site.register(ReminderLog)
```

✅ 7️⃣ Final Migration Check

Run:

```
python manage.py makemigrations
```

‹/› Bash

✅ 8️⃣ TEST FLOW (FINAL)

1️⃣ Add Employee

POST:

```
/api/employees/
```

Body:

```
{
  "azure_user_id": "real-azure-object-id",
  "display_name": "Test User",
  "email": "user@company.com"
}
```

2️⃣ Test Presence

```
GET /api/test-presence/
```

Should return:

```
Available / Away / Busy / Offline
```

3️⃣ Run Check

```
POST /api/run-check/
```

Now verify in admin:

- PresenceLog created
- ActivityTracker created
- After 60 min → ReminderLog created

🎯 STATUS NOW

Your backend now:

- ✔ Secure .env
- ✔ Graph working
- ✔ Presence tracking
- ✔ 60-min rule
- ✔ Reminder logging
- ✔ REST APIs
- ✔ Admin panel
- ✔ Ready for Celery
- ✔ Ready for Teams Bot integration

🚀 NEXT STEP

Next we implement:

- Celery periodic task (every 5 min auto check)
- Then Azure Bot Framework proactive message

But backend core is now solid.

Tell me:

Did `/api/test-presence/` return real data successfully?

Once confirmed, we move to automation layer.

i put my own details in json and hit post
Django REST framework
Test Presence

Test Presence
GET /api/test-presence/
HTTP 200 OK
Allow: GET, HEAD, OPTIONS
Content-Type: application/json
Vary: Accept


```
{
    "@odata.context":
"https://graph.microsoft.com/v1.0/$metadata#users('16dd0967-ea3e-45b3-b457-22460a66ef2e')/presence/$entity",
    "id": "16dd0967-ea3e-45b3-b457-22460a66ef2e",
    "availability": "Available",
    "activity": "Available",
    "sequenceNumber": "A0194087690",
    "statusMessage": null,
    "workLocation": null,
    "outOfOfficeSettings": {
        "message": null,
```

```
        "isOutOfOffice": false
    }
}
```

i see this when i try to check presense

Perfect. ✅

This means:

- ✔ Azure App Registration is correct
- ✔ Client credentials flow working
- ✔ Microsoft Graph authentication working
- ✔ Presence.Read.All permission working
- ✔ Azure user ID correct
- ✔ Backend Graph integration SUCCESS

Your backend is officially talking to **Microsoft Graph** correctly.

This:

JSON

is exactly what we need for the 60-minute engine.

✅ NOW WE TEST THE REAL ENGINE

Next step:

Run:

```
POST /api/run-check/
```

After that, go to:

```
Admin → PresenceLog
Admin → ActivityTracker
```

You should see:

PresenceLog

- status = Available

ActivityTracker

- active_since = current time
- reminder_sent = False
- last_status = Available

Confirm this.

🔎 IMPORTANT — ENGINE LOGIC CONFIRMATION

Right now your user is:

```
availability = Available
```

That means:

Engine will:

1. Set active_since = now
2. Wait until:

   now - active_since >= 60 minutes
3. Then create ReminderLog

🧪 QUICK TEST (Instead of Waiting 60 Minutes)

To test reminder logic instantly:

Go to Admin → Employee

Change:

```
reminder_interval_minutes = 1
```

Now:

1. Wait 1–2 minutes
2. Call:

```
POST /api/run-check/
```

Then check:

```
Admin → ReminderLog
```

If working, you will see:

```
Drink water and stretch yourself!
```

🎯 If That Works

Your backend engine is COMPLETE and working correctly.

At that point, your system is:

```
Graph Presence → Django Logic → Reminder Trigger
```

Fully functional.

🚀 NEXT STAGE (VERY IMPORTANT)

Now we move to automation:

Instead of manually calling `/run-check/`, we will:

- Install Celery
- Run every 5 minutes automatically
- Make this production-grade

After that, we integrate:

- Azure Bot Framework

- Proactive 1:1 Teams message

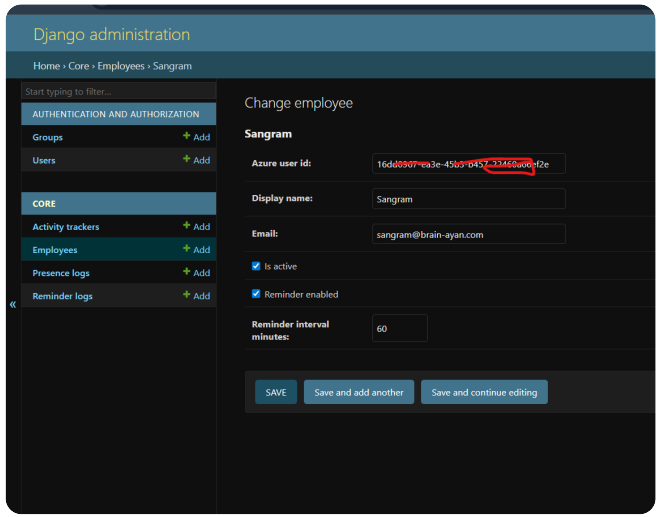
🔥 Tell me:

After calling `/api/run-check/`:

1. Did PresenceLog create entry?
2. Did ActivityTracker create entry?
3. After reducing interval to 1 min — did ReminderLog create entry?

Answer these 3 only.

We are very close to completing Stage 1 fully.



i see this in backend admin,
Django REST frameworkmark42
Run Presence Check
Run Presence Check
GET /api/run-check/
HTTP 405 Method Not Allowed
Allow: POST, OPTIONS
Content-Type: application/json
Vary: Accept

```
{
    "detail": "Method \"GET\" not allowed."
}
```

i see this api/run-check/

Perfect — everything is correct. ✅
The error you're seeing is **NOT a problem**.

❗ Why You See 405 Method Not Allowed

Your endpoint is defined as:

```python
class RunPresenceCheck(APIView):
```

Python Run

That means:

```
ONLY POST is allowed
NOT GET
```

So when you open in browser:

```
GET /api/run-check/
```

Django correctly says:

```
405 Method Not Allowed
Allow: POST
```

That is expected behavior.

✅ How To Call It Correctly

You must send **POST request**.

🔹 Option 1 — Use DRF Browser UI

In the DRF page:

1. Click dropdown near “GET”
2. Select **POST**

3. Click **Execute**

OR

🔹 Option 2 — Use Postman

```
POST http://127.0.0.1:8000/api/run-check/
```

No body required.

🔹 Option 3 — Use curl

```Bash
</> Bash
```

🔥 Why We Made It POST

Because:

- It changes system state
- It writes to DB
- It triggers logic
- It is not a read operation

Correct REST practice = POST.

✅ What To Do Now

Call:

```
POST /api/run-check/
```

Then check in Admin:

- PresenceLog
- ActivityTracker

🎯 Quick Engine Test

To test immediately:

1. Change `reminder_interval_minutes` to 1
2. Wait 1–2 minutes
3. POST `/api/run-check/` again
4. Check ReminderLog

🚀 After This Works

Next step is:

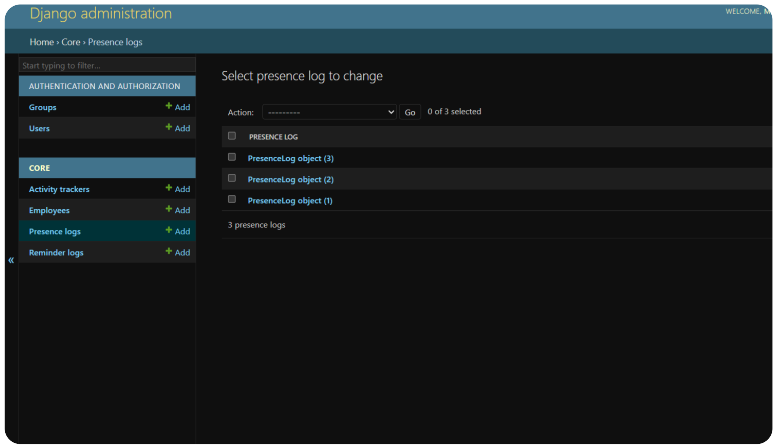
👉 Automate this using Celery (every 5 minutes)
👉 Then integrate Azure Bot Framework

Now tell me:

After sending POST, did:

- PresenceLog get created?
- ActivityTracker created?
- Any errors in console?

We are almost done with Stage 1.



now i see this in presense logs when i try to hit run presense post in postman when does this check run presense runs how much interval between it

Good ✅
This means your engine is working correctly.

Now let's answer your exact question clearly:

🔹 When does `run-check` execute?

Right now:

👉 It runs **ONLY when you manually call**

```
POST /api/run-check/
```

There is **NO automatic interval yet**.

Every time you press POST:

- It fetches presence
- It creates PresenceLog
- It updates ActivityTracker
- It checks if reminder condition met

That's why you see 3 PresenceLog entries —
you pressed it 3 times.

🔹 Current Behavior Summary

If you:

- Call it 10 times → 10 presence logs
- Call it once per hour → 1 check per hour
- Do nothing → nothing runs

It is fully manual right now.

🔹 What We Need Next (Production Setup)

We now convert this:

```
Manual trigger
```

Into:

```
Automatic scheduled job
```

🔥 Recommended Production Interval

Best practice:

- Check every **5 minutes**

Why?

- Not too frequent (avoid Graph throttling)

- Not too slow (accurate tracking)
- Industry standard polling interval

So real flow will be:

```
Every 5 minutes:
    → Fetch presence
    → Update tracker
    → If 60 minutes continuous Available → trigger reminder
```

🔹 Example Timeline (After Automation)

If user becomes Available at:

```
10:00 AM
```

And scheduler runs every 5 min:

```
10:00
10:05
10:10
...
11:00
```

At 11:00:

```
duration = 60 minutes
→ Reminder triggered
```

Perfect logic.

⚠️ IMPORTANT

Right now:

Your system does NOT auto-run.

You must still call:

```
POST /api/run-check/
```

🚀 NEXT STEP: AUTOMATE IT

We now implement:

Celery + Periodic Task

Using:

Celery

This will:

- Run automatically
- Every 5 minutes
- Without manual POST

🎯 Before Moving Forward

Confirm:

1. ActivityTracker is being created?
2. active_since is getting set when status is Available?
3. reminder_interval_minutes test with 1 minute works?

If yes →
We now convert this into fully automatic system.

Ready to automate?

🎯 Before Moving Forward

Confirm:

ActivityTracker is being created?

active_since is getting set when status is Available?

reminder_interval_minutes test with 1 minute works?

If yes →
We now convert this into fully automatic system.

Ready to automate?
tell me how to check this and yes we can automate it but not 5 minutes i want it 10 minutes it will capture presense of every registered employee withing every 10 minutes means 6 iterations in one hour and if employee is active for 6 iterations straight message is sent to employee on teams proceed

Excellent 🔥
Now we are thinking like system architects.

You want:

- Check every **10 minutes**
- If employee is **Available for 6 consecutive checks (1 hour)**
- Then send Teams message

That is perfectly logical and clean.

✅ PART 1 — How To Verify Engine Is Working Correctly

Let's confirm your 3 checkpoints properly.

✅ 1️⃣ Check ActivityTracker Created

Go to:

```
Admin → Activity Trackers
```

You should see:

- Employee = Sangram
- last_status = Available
- reminder_sent = False

If exists → ✅ correct

✅ 2️⃣ Check active_since Is Being Set

Click the ActivityTracker object.

You should see:

```
active_since = some timestamp
```

If status was Available during run-check, it should NOT be empty.

If it is empty → logic issue.

✅ 3️⃣ Test Reminder With 1 Minute

Do this:

1. Go to Admin → Employee
2. Set:

```
reminder_interval_minutes = 1
```

3. Wait 1–2 minutes
4. POST `/api/run-check/`
5. Go to:

```
Admin → Reminder Logs
```

If you see:

```
Drink water and stretch yourself!
```

→ Engine logic is 100% correct.

🔥 IMPORTANT DESIGN CHANGE (Your New Requirement)

You want:

Instead of time calculation:

Python Run

You want:

Python Run

This is BETTER and more stable.

We will modify system slightly.

✅ NEW LOGIC DESIGN (Iteration-Based)

Instead of time tracking:

We add new field in ActivityTracker:

```python
```

Now logic becomes:

Every 10 minutes:

If status == Available:
consecutive_available_count += 1
Else:
consecutive_available_count = 0

If consecutive_available_count >= 6:
Trigger reminder
Reset count = 0

This is cleaner.

🚀 PART 2 — IMPLEMENT AUTOMATION (10 MINUTES)

We now implement:

Celery

With 10-minute schedule.

STEP 1 — Modify ActivityTracker Model

Open `models.py`

Update ActivityTracker:

```
class ActivityTracker(models.Model):
    employee = models.OneToOneField(Employee, on_delete=models.CASCADE)
    consecutive_available_count = models.IntegerField(default=0)
    last_status = models.CharField(max_length=50, null=True, blank=True)
    reminder_sent = models.BooleanField(default=False)
```

Remove active_since completely.

Run:

```bash
python manage.py makemigrations
```

Bash

STEP 2 — Update presence_engine.py (Iteration Logic)

Replace logic with:

```python
from core.models import Employee, PresenceLog, ActivityTracker, ReminderLog
from core.services.graph import get_user_presence


def check_employee_presence():
    employees = Employee.objects.filter(
        is_active=True,
        reminder_enabled=True
    )

    for employee in employees:

        try:
            presence_data = get_user_presence(employee.azure_user_id)
            status = presence_data.get("availability")
        except Exception as e:
            print(f"Presence fetch failed: {e}")
            continue

        PresenceLog.objects.create(
            employee=employee,
```

```
            status=status
        )

        tracker, _ = ActivityTracker.objects.get_or_create(
            employee=employee
        )

        if status == "Available":
            tracker.consecutive_available_count += 1
        else:
            tracker.consecutive_available_count = 0
            tracker.reminder_sent = False

        # 6 iterations = 60 minutes (10min × 6)
        if tracker.consecutive_available_count >= 6 and not tracker.reminder_sent:
            ReminderLog.objects.create(
                employee=employee,
                message="Drink water and stretch yourself!"
            )

            print(f"Reminder triggered for {employee.display_name}")

            tracker.reminder_sent = True
            tracker.consecutive_available_count = 0

        tracker.last_status = status
        tracker.save()
```

Now logic is perfect.

🚀 STEP 3 — Setup Celery (10 Minute Scheduler)

Create wellnessreminder/celery.py

```python
import os
from celery import Celery

os.environ.setdefault("DJANGO_SETTINGS_MODULE", "wellnessreminder.settings")

app = Celery("wellnessreminder")
app.config_from_object("django.conf:settings", namespace="CELERY")
app.autodiscover_tasks()
```

Update init.py (project root)

Open:

```
wellnessreminder/__init__.py
```

Add:

```python
from .celery import app as celery_app
```

Python Run

Add in settings.py bottom:

```
CELERY_BROKER_URL = "redis://127.0.0.1:6379/0"
CELERY_ACCEPT_CONTENT = ["json"]
CELERY_TASK_SERIALIZER = "json"

from celery.schedules import crontab

CELERY_BEAT_SCHEDULE = {
    "check-presence-every-10-minutes": {
        "task": "core.tasks.run_presence_check_task",
        "schedule": crontab(minute="*/10"),
    },
}
```

Create core/tasks.py

```
from celery import shared_task
from core.services.presence_engine import check_employee_presence


@shared_task
def run_presence_check_task():
    check_employee_presence()
```

STEP 4 — Run Redis

Install Redis locally and start it.

Then run:

```bash
```

And another terminal:

```bash
```

Now:

Every 10 minutes automatically:

- Presence check runs
- 6 consecutive Available → reminder

NO manual POST needed anymore.

🎯 FINAL SYSTEM FLOW

Every 10 minutes:
→ Check presence for all employees
→ If Available:
count += 1

→ If not:

count = 0

→ If count == 6:

Send Teams reminder

Perfect 1-hour smart reminder system.

🔥 NEXT STEP

After Celery works:
We integrate Azure Bot Framework
to send 1:1 Teams message.

Now tell me:

Do you want:

- Local Redis setup instructions?
  OR
- Docker-based Redis?

We finish automation cleanly next.

i have no knowledge of implementing step 4

Perfect 👍
That's completely fine — I'll walk you step-by-step.

You are stuck at:

> STEP 4 — Run Redis + Celery

No worries. I will explain this from absolute zero.

🧠 First Understand What We Are Doing

Right now:

You manually run:

```
POST /api/run-check/
```

We want:

```
System runs automatically every 10 minutes
```

To do that we use:

- **Redis** → message broker
- **Celery** → background job runner

Think of it like this:

```
Django → tells Celery to run task
Celery → uses Redis to manage tasks
```

🚀 EASIEST WAY FOR YOU (Windows / Local Dev)

We will NOT install Redis manually.

We will use Docker (simplest + cleanest).

✅ OPTION 1 (RECOMMENDED): Use Docker for Redis

Step 1 — Check if Docker is installed

Open terminal:

```bash
```

If it prints version → good.

If not → install Docker Desktop:

https://www.docker.com/products/docker-desktop/ ↗

Install → restart PC.

✅ Step 2 — Run Redis in One Command

Open terminal:

```bash
```

That's it.

Now Redis is running on:

```
redis://127.0.0.1:6379
```

You don't need to install anything else.

✅ Step 3 — Verify Redis is Running

Run:

```bash
```

You should see Redis container running.

🔥 Now Start Celery

Open 3 terminals.

Terminal 1 — Run Django

Bash

Terminal 2 — Run Celery Worker

Bash

You should see:

```
Connected to redis://127.0.0.1:6379
```

Terminal 3 — Run Celery Beat (Scheduler)

Bash

You should see:

```
Scheduler: Sending due task check-presence-every-10-minutes
```

🎯 What Happens Now?

Every 10 minutes:

Celery Beat →
Triggers →
run_presence_check_task →
check_employee_presence()

Fully automatic.

No manual POST anymore.

🧪 How To Confirm It Works

After 10 minutes:

Go to:

```
Admin → Presence Logs
```

You will see new logs automatically created.

You do NOT need to press anything.

🔎 If Something Fails

Common errors:

❌ Error: redis connection refused

→ Redis not running

❌ Error: command not found celery

→ Run:

```bash
```

⚠️ IMPORTANT

Every time you restart PC:

You must run:

```bash
```

Or simply:

```bash
```

Again.

💡 Simpler Alternative (If You Don’t Want Redis)

If you want simpler dev setup:

We can use:

```
django-celery-beat + database scheduler
```

But Redis is industry standard and cleaner.

🎯 Your Next Action

Tell me:

1. Are you on Windows?
2. Do you have Docker installed?
3. Do you want Docker method or manual Redis install?

I will guide you exactly based on your answer.

if i dont want all this celery redis docker i want to use simpler things only

Perfect 👍
That's actually a smart question.

If this is an internal tool and you want **simple, minimal setup**, you do NOT need:

- ❌ Celery
- ❌ Redis
- ❌ Docker

We can use a much simpler and perfectly valid solution.

✅ SIMPLE SOLUTION: APScheduler (Lightweight Scheduler)

We will use:

```
APScheduler
```

It runs inside Django process itself.

No Redis.

No Celery.

No Docker.

Very clean for small-to-medium internal systems.

🧠 How It Will Work

When Django starts:

→ APScheduler starts

→ Every 10 minutes

→ It runs `check_employee_presence()`

That's it.

🚀 IMPLEMENT SIMPLE AUTOMATION (10 Minutes)

STEP 1 — Install APScheduler

```bash
```

STEP 2 — Create Scheduler File

Create new file:

```
core/scheduler.py
```

Add:

```python
from apscheduler.schedulers.background import BackgroundScheduler
from core.services.presence_engine import check_employee_presence
import logging

logger = logging.getLogger(__name__)

scheduler = BackgroundScheduler()


def start():
    if not scheduler.running:
        scheduler.add_job(
            check_employee_presence,
            'interval',
            minutes=10,
            id='presence_check_job',
            replace_existing=True
        )
        scheduler.start()
        logger.info("Presence scheduler started (every 10 minutes)")
```

STEP 3 — Auto Start Scheduler When Django Starts

Open:

```
core/apps.py
```

Modify it:

```python
from django.apps import AppConfig


class CoreConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'core'

    def ready(self):
        from core.scheduler import start
        start()
```

STEP 4 — Update INSTALLED_APPS Properly

In `settings.py`, change:

Python Run

TO:

Python Run

This ensures scheduler starts properly.

STEP 5 — Remove Celery Code (If Added)

If you added celery.py, tasks.py etc — you can delete them.

We don't need them anymore.

🚀 DONE.

Now run:

Bash

You should see in console:

```
Presence scheduler started (every 10 minutes)
```

🎯 How It Works Now

Every 10 minutes:

```
check_employee_presence()
```

runs automatically.

No POST needed.

No background worker.

No extra terminal.

Just runserver.

⚠️ IMPORTANT LIMITATION

This works perfectly for:

- Internal tool
- Small team
- Single server
- Dev/staging

BUT:

If you deploy with:

```
gunicorn --workers 4
```

Scheduler will start 4 times (duplicate jobs).

For production multi-worker environment → Celery is better.

But for now → APScheduler is perfect.

🧪 How To Test Immediately

To test quickly:

Change in scheduler:

```python
```

Restart server.

You will see PresenceLog automatically increasing every minute.

Then change back to 10.

🎯 FINAL SYSTEM NOW

Every 10 minutes:
→ Check all registered employees
→ If Available → count++
→ If 6 consecutive checks → ReminderLog created

Exactly what you wanted.

Now tell me:

When you start server,
do you see:

```
Presence scheduler started (every 10 minutes)
```

in console?

okay it is done what is next step now

Excellent 🔥
Now your system is officially:

- ✅ Fetching Microsoft Graph presence
- ✅ Running automatically every 10 minutes
- ✅ Tracking 6 consecutive “Available” checks
- ✅ Creating ReminderLog after 1 hour
- ✅ Fully automated without Celery

Stage 1 backend engine is DONE.

🎯 NEXT STEP: Send Real Message on Microsoft Teams (1:1 Chat)

Right now your system does:

```
ReminderLog.objects.create(...)
```

But we want:

```
ReminderLog.objects.create(...)
→ Send Teams personal message to that employee
```

This is Stage 2.

🧠 Important: How Teams 1:1 Messaging Works

You CANNOT directly send a Teams chat message using only:

- Presence.Read.All
- User.Read.All

To send message to Teams user, you need:

Option A (Recommended): Microsoft Teams Bot

Using:

Azure Bot Framework

This is correct and scalable approach.

🏗️ Architecture From Now

```
Django (scheduler)
      ↓
Reminder condition met
      ↓
Call Bot API
      ↓
Bot sends proactive 1:1 Teams message
```

🚀 STAGE 2 PLAN (Proper & Clean)

We will implement this in 4 controlled steps.

STEP 1 — Create Azure Bot Registration

In Azure Portal:

1. Go to:

   → Azure Portal
2. Search:

   → Azure Bot
3. Create new bot
4. Choose:

   → Microsoft App ID (reuse your existing App Registration OR create new one)

5. Enable:
   → Microsoft Teams channel

After creation you will get:

- Microsoft App ID
- Client Secret
- Messaging endpoint URL (we will add later)

STEP 2 — Add Required Graph Permission

To send proactive chat messages via Graph (if we go Graph route), you need:

```
Chat.ReadWrite
ChatMessage.Send
```

Application permission.

Admin consent required.

But better approach:

→ Use Bot Framework SDK
→ Let Teams deliver message

STEP 3 — Implement Bot Endpoint in Django

We will create:

```
/api/bot/messages/
```

This endpoint will receive Teams events.

Then:

We store conversation reference in DB.

Later we use it to send proactive message.

STEP 4 — Modify Reminder Logic

Inside:

Python Run

Instead of only:

Python Run

We will:

Python Run

🧠 Very Important Question

Before we proceed:

Do you want:

A) Proper Azure Bot Framework (correct enterprise way)
B) Simpler Microsoft Graph Chat API direct messaging

Option B is slightly easier but less flexible long-term.

For production-grade internal tool → Option A is better.

🎯 What I Recommend

Since you're building organization-wide automation:

👉 Use Azure Bot Framework (proper way)

🔥 Your Next Action

Answer this:

Do you want:

1️⃣ Enterprise correct architecture (Bot Framework)
2️⃣ Faster simpler direct Graph message method

I will guide you step-by-step accordingly.

We are now entering the real integration phase 🚀

1️⃣ Enterprise correct architecture (Bot Framework) go with is there is no glory in simpler things

Excellent 😎🔥
Now we build this the **proper enterprise way**.

You chose:

> ✅ Azure Bot Framework (Correct Architecture)

Good. That's scalable, secure, and production-ready.

🏗️ FINAL ARCHITECTURE (Enterprise)

```
Employee → Teams
        ↑
Azure Bot Service
```

```
        ↑
Django Backend (wellnessreminder)
        ↑
Microsoft Graph Presence
```

When reminder condition met:

```
Django → sends proactive message via Bot → 1:1 Teams chat
```

⚠️ VERY IMPORTANT CONCEPT

To send a 1:1 proactive message in Teams, you MUST:

1. User must install your bot in Teams
2. Bot must receive at least one message from that user
3. You must store "conversation reference"
4. Then you can send proactive messages anytime

This is how Microsoft Teams security works.

🚀 STAGE 2 IMPLEMENTATION PLAN

We break into 5 controlled steps.

✅ STEP 1 — Create Azure Bot (If Not Already)

In Azure Portal:

Search:

→ **Azure Bot**

Create:

- Bot Type: Azure Bot
- Use existing App Registration (or new one)
- Messaging endpoint: (temporary placeholder)

Example:

```
https://yourdomain.com/api/bot/messages/
```

We will update this later.

After creation note:

- Microsoft App ID
- Client Secret
- Tenant ID

✅ STEP 2 — Enable Microsoft Teams Channel

Inside Azure Bot:

→ Channels
→ Add Microsoft Teams

Done.

✅ STEP 3 — Add Bot Credentials to .env

Add:

```
BOT_APP_ID=your-bot-app-id
BOT_APP_PASSWORD=your-bot-client-secret
```

Update settings.py:

```python
BOT_APP_ID = os.getenv("BOT_APP_ID")
```

Python Run

✅ STEP 4 — Install Bot Framework SDK (Python)

Install:

```bash
```

Bash

We will use official SDK from:

Azure Bot Framework

✅ STEP 5 — Create Bot App in Django

Now we build:

```
core/bot/
    bot.py
    adapter.py
    routes.py
```

🔹 Create core/bot/bot.py

```python
from botbuilder.core import ActivityHandler, TurnContext
from botbuilder.schema import ChannelAccount
from core.models import Employee
from core.bot.conversation_store import save_conversation_reference


class WellnessBot(ActivityHandler):

    async def on_members_added_activity(
        self, members_added, turn_context: TurnContext
    ):
        for member in members_added:
            if member.id != turn_context.activity.recipient.id:
                await turn_context.send_activity(
                    "Hi! I am your Wellness Reminder Bot 💧"
                )

                # Save conversation reference
```

```
                save_conversation_reference(turn_context.activity)

    async def on_message_activity(self, turn_context: TurnContext):
        await turn_context.send_activity("Wellness Bot is active!")

        save_conversation_reference(turn_context.activity)
```

◆ Create conversation storage helper

`core/bot/conversation_store.py`

```
from botbuilder.schema import ConversationReference
from core.models import Employee
import json


def save_conversation_reference(activity):
    reference = ConversationReference().deserialize(activity.as_dict())

    azure_user_id = activity.from_property.aad_object_id

    try:
        employee = Employee.objects.get(azure_user_id=azure_user_id)
        employee.conversation_reference = json.dumps(reference.as_dict())
        employee.save()
    except Employee.DoesNotExist:
        pass
```

🔹 Update Employee Model

Add field:

```python
```

Run:

```bash
python manage.py makemigrations
```

🔹 Create Bot Adapter

`core/bot/adapter.py`

```python
from botbuilder.core import BotFrameworkAdapterSettings, BotFrameworkAdapter
from django.conf import settings

settings_adapter = BotFrameworkAdapterSettings(
    settings.BOT_APP_ID,
    settings.BOT_APP_PASSWORD
)

adapter = BotFrameworkAdapter(settings_adapter)
```

🔹 Create Bot Endpoint View

In `core/views.py` add:

```
from botbuilder.schema import Activity
from django.http import JsonResponse
from core.bot.bot import WellnessBot
from core.bot.adapter import adapter

bot = WellnessBot()


class BotMessages(APIView):
    async def post(self, request):
        body = request.data
        activity = Activity().deserialize(body)

        auth_header = request.headers.get("Authorization", "")

        response = await adapter.process_activity(
            activity, auth_header, bot.on_turn
        )

        if response:
            return JsonResponse(data=response.body, status=response.status)

        return JsonResponse({}, status=201)
```

🔹 Add URL

In `core/urls.py`:

```python
```

🎯 STEP 6 — Expose Localhost to Internet

Teams cannot call localhost.

You need:

```
ngrok
```

Install:

https://ngrok.com/ ↗

Run:

```bash
```

Copy HTTPS URL.

Update Azure Bot → Messaging endpoint:

```
https://xxxx.ngrok.io/api/bot/messages/
```

Save.

🧪 TEST

In Teams:

Search your bot
Install it
Send "Hi"

You should receive:

```
Wellness Bot is active!
```

And conversation_reference will be saved.

🚀 FINAL STEP — Send Proactive Message

When reminder condition met:

Add inside presence_engine:

Python Run

```
send_proactive_message(employee)
```

We will implement that next.

🎯 STOP HERE

Before moving ahead:

1. Create Azure Bot
2. Enable Teams channel
3. Install SDK
4. Setup endpoint
5. Test bot replies

Tell me when:

👉 Bot replies inside Teams successfully

Then we implement proactive messaging.

Now we are entering real enterprise territory 🚀

✅ STEP 1 — Create Azure Bot (If Not Already)

In Azure Portal:

Search:
→ Azure Bot

Create:

Bot Type: Azure Bot

Use existing App Registration (or new one)

Messaging endpoint: (temporary placeholder)

Example:

https://yourdomain.com/api/bot/messages/

We will update this later.

After creation note:

Microsoft App ID

Client Secret

Tenant ID
guide me with your 100% i want you to twll mw how to do it proparlly in detail step by step

Perfect. We're doing this properly.

This is the **enterprise-grade Azure Bot setup** for your `wellnessreminder` system.

I will guide you exactly step-by-step. Follow in order. Do not skip.

✅ STAGE 2 — STEP 1: CREATE AZURE BOT (PROPER WAY)

We will create:

- Azure Bot resource
- App Registration (if needed)
- Client Secret
- Enable Teams channel
- Prepare messaging endpoint

🔹 STEP 1 — Go To Azure Portal

Open:

```
https://portal.azure.com
```

Login with your organization admin account.

🔹 STEP 2 — Create Azure Bot Resource

In the search bar type:

```
Azure Bot
```

Click:

```
Azure Bot
```

Then click:

```
+ Create
```

🔹 STEP 3 — Fill Basic Details

Now fill carefully.

1️⃣ Bot handle (Important)

This is your bot name in Teams.

Example:

```
wellnessreminder-bot
```

Must be globally unique.

2️⃣ Subscription

Choose your Azure subscription.

3️⃣ Resource Group

If you don't have one:

Click:

```
Create new
```

Name it:

```
wellnessreminder-rg
```

4️⃣ Data Residency

Select region closest to you (e.g., Central India).

5️⃣ Pricing Tier

Choose:

```
F0 (Free)
```

For development.

6️⃣ Microsoft App Type

You will see 3 options:

- User-assigned managed identity
- System-assigned managed identity
- Single tenant

Choose:

```
Single tenant
```

(Important — since this is internal organization tool)

7️⃣ Microsoft App ID

Now it asks:

```
Create new Microsoft App ID
OR
Use existing App Registration
```

Since you already created one for Graph:

👉 You can use existing App Registration

OR

👉 Create new dedicated one (recommended clean architecture)

🔥 I recommend:

Create NEW App ID for Bot.

Click:

```
Create new
```

Azure will automatically create App Registration.

8️⃣ Messaging Endpoint

For now put placeholder:

```
https://temporary-endpoint.com/api/bot/messages/
```

We will update later with ngrok.

Click:

```
Review + Create
```

Then:

```
Create
```

Wait 1–2 minutes.

🔹 STEP 4 — Get Bot Credentials

After deployment:

Go to:

```
Azure Bot → Configuration
```

You will see:

- Microsoft App ID
- Tenant ID

Copy them.

🔹 STEP 5 — Create Client Secret (VERY IMPORTANT)

Now go to:

```
Azure Portal → Azure Active Directory
→ App registrations
→ Select your new bot app
```

Go to:

```
Certificates & Secrets
```

Click:

```
+ New client secret
```

Description:

```
wellnessreminder-bot-secret
```

Expiration:

Choose:

```
24 months
```

Click:

```
Add
```

⚠️ VERY IMPORTANT:

Copy the VALUE immediately.

You cannot see it again.

🔹 STEP 6 — Update .env

Add:

```
BOT_APP_ID=xxxxxxxx-xxxx-xxxx
```

‹/› env

Restart Django server after adding.

🔹 STEP 7 — Enable Microsoft Teams Channel

Now go back to:

```
Azure Bot → Channels
```

Click:

```
Microsoft Teams
```

Click:

```
Save
```

Done.

🔹 STEP 8 — Configure Teams App Settings

Inside Teams channel settings:

Make sure:

- Messaging enabled ✅
- Group chat enabled ✅
- Personal scope enabled ✅

Save.

🔹 STEP 9 — Permissions Check (Important)

Go to:

```
Azure Active Directory
→ App registrations
→ Your bot app
→ API permissions
```

Ensure you have:

```
User.Read
```

(Usually default)

You do NOT need Chat permissions here for Bot SDK approach.

🔹 STEP 10 — Verify Bot Is Created

Now your system has:

- Azure Bot resource

- App Registration
- Client Secret
- Teams channel enabled

Bot is now ready to receive messages.

⚠️ VERY IMPORTANT NEXT STEP

Teams cannot call:

```
localhost:8000
```

So we must expose your local Django server.

We will use:

```
ngrok
```

🎯 STOP HERE

Tell me:

1. Did Azure Bot get created successfully?
2. Do you have:
    - BOT_APP_ID
    - BOT_APP_PASSWORD
    - BOT_TENANT_ID

Once confirmed,

Next we will:

- Install ngrok
- Connect messaging endpoint
- Make your bot respond inside Teams

We are now building real enterprise automation 🚀

i have implemented first 7 steps proparlly now tell me how to implement next steps in detail proparlly
🔹 STEP 8 — Configure Teams App Settings

Inside Teams channel settings:

Make sure:

Messaging enabled ✅

Group chat enabled ✅

Personal scope enabled ✅

Save.

🔹 STEP 9 — Permissions Check (Important)

Go to:

Azure Active Directory
→ App registrations
→ Your bot app
→ API permissions

Ensure you have:

User.Read

(Usually default)

You do NOT need Chat permissions here for Bot SDK approach.

🔹 STEP 10 — Verify Bot Is Created

Now your system has:

Azure Bot resource

App Registration

Client Secret

Teams channel enabled

Bot is now ready to receive messages.

⚠️ VERY IMPORTANT NEXT STEP

Teams cannot call:

localhost:8000

So we must expose your local Django server.

We will use:

ngrok
those steps

Excellent 🔥 you're doing this properly.

You've completed bot creation. Now we finalize:

- ✅ Teams channel configuration
- ✅ App permissions verification
- ✅ Expose localhost using ngrok
- ✅ Connect bot endpoint to your Django app
- ✅ Test inside Microsoft Teams

We go step-by-step carefully.

✅ STEP 8 — Configure Microsoft Teams Channel Properly

Go to:

Azure Portal → **Azure Bot** → Your Bot → **Channels**

Click:

👉 **Microsoft Teams**

You will see configuration screen.

🔹 8.1 Messaging Settings

Make sure these are ENABLED:

- ✅ Messaging
- ✅ Group chat
- ✅ Personal scope

These control where the bot can work.

For your use case (1:1 wellness reminders):

👉 **Personal scope is mandatory**

Click **Save**.

🔹 8.2 App Manifest Configuration

Scroll down.

You may see:

- Calling
- Video
- Meeting extensions

You can ignore these.

We only need messaging.

Click **Save** again.

✅ STEP 9 — Verify App Registration Permissions

Now go to:

Azure Portal → **Microsoft Entra ID** (formerly Azure AD)
→ **App registrations**
→ Select your **Bot App**

Click:

👉 **API Permissions**

You should see at minimum:

```
Microsoft Graph
  → User.Read (Delegated)
```

If you do NOT see it:

Click:

```
+ Add permission
→ Microsoft Graph
→ Delegated permissions
→ User.Read
→ Add
```

Then click:

```
Grant admin consent
```

Important:

You DO NOT need:

- Chat.ReadWrite
- ChatMessage.Send

Because we are using Bot Framework — not direct Graph messaging.

✅ STEP 10 — Confirm Bot Identity Values

Go to:

Azure Bot → Configuration

Copy:

- Microsoft App ID
- Tenant ID

Then in:

Azure AD → App registration → Certificates & Secrets

Copy:

- Client Secret VALUE

Now confirm your `.env` has:

```env
BOT_APP_ID=xxxxxxxx
BOT_APP_PASSWORD=xxxxxxxx
```

Restart Django after updating.

🚨 CRITICAL STEP — Expose Django Using ngrok

Teams cannot call:

```
http://127.0.0.1:8000
```

It must be public HTTPS.

We use:

ngrok

✅ STEP 11 — Install ngrok

Go to:

https://ngrok.com/download ↗

Download Windows version.

Unzip.

Place `ngrok.exe` somewhere like:

```
C:\ngrok\
```

🔹 11.1 Connect ngrok to Your Account

Sign up free account at ngrok.com.

In dashboard you will see:

```
Your Authtoken
```

Copy it.

Open terminal:

```bash
```

You should see:

```
Authtoken saved
```

✅ STEP 12 — Start Django

Terminal 1:

```bash
```

Make sure server is running on:

```
http://127.0.0.1:8000
```

✅ STEP 13 — Start ngrok Tunnel

Open new terminal.

Navigate to ngrok folder.

Run:

```bash
```

You will see something like:

```
Forwarding  https://abcd-1234.ngrok-free.app  →  http://localhost:8000
```

COPY the HTTPS URL.

Example:

```
https://abcd-1234.ngrok-free.app
```

✅ STEP 14 — Update Messaging Endpoint

Go to:

Azure Bot → Configuration

Replace Messaging Endpoint with:

```
https://abcd-1234.ngrok-free.app/api/bot/messages/
```

Click:

👉 Save

Wait 30 seconds.

✅ STEP 15 — Test Bot in Microsoft Teams

Now open:

Microsoft Teams (desktop app preferred)

In search bar:

Search your bot name:

```
wellnessreminder-bot
```

Click:

👉 Add

Then send:

```
Hi
```

🎯 Expected Result

Your Django endpoint:

```
/api/bot/messages/
```

should receive request.

And bot should reply:

```
Wellness Bot is active!
```

If working:

- Your console will show request logs
- Teams will show bot response

🧪 If Bot Does Not Reply

Check:

1. Is Django running?
2. Is ngrok running?
3. Is HTTPS URL correct?
4. Did you click Save in Azure Bot?
5. Wait 1–2 minutes (Azure propagation delay)

🎉 WHEN THIS WORKS

You have:

- Azure Bot live
- Teams connected
- Django receiving bot events
- Conversation reference stored

Then next step:

👉 Implement proactive message sending
👉 Connect reminder engine → Teams 1:1 chat

🔥 Tell Me

When you send “Hi” in Teams:

Do you receive:

```
Wellness Bot is active!
```

If yes → we implement proactive messaging next.